## Level 2
## Hackathon – Fundamentals of Web Development

| | |
|---|---|
| **Student Name** | YOKESHWARAN D |
| **Register Number** | 25132041802521048 |
| **Department** | BSC, COMPUTER SCIENCE |
| **ID** | F78da7dc32e3e70a18e424868977253f |
| **College** | Srimath Sivagnana Balaya Swamigal Tamil,Arts,Science College-Mailam |
| **Date of Submission** | 29.10.2026 |

**Git-Hub Link:**

https://github.com/yokesyokes532-svg/student-Attendance-Trackers

**Project Summary**

The Student Attendance Tracker is a browser-based application for maintaining student details and recording daily attendance. It allows the user to add, edit, and delete student records, mark attendance for a selected date, search and sort the student list, and review each student's attendance history and percentage. The application is built with HTML, CSS, and JavaScript. It stores records in the browser's Local Storage, so the data remains available in the same browser on the same device.

### 1. Problem Statement

Recording attendance manually in registers or separate files takes time and makes it difficult to find student details, calculate attendance percentages, and identify students with low attendance. Records may also become difficult to update and review. This project provides a single, simple web application in which student details and daily attendance can be managed together. It helps reduce repetitive work, makes attendance records easier to search, and calculates attendance percentages automatically.

### 2. Objectives

- To add, edit, and delete student records containing the student's name, unique student ID, and department.
- To record each student's attendance status for a selected date.
- To apply the same attendance status to all registered students when required.
- To calculate each student's attendance percentage and the overall attendance percentage from the saved attendance records.
- To search student records by name, student ID, or department, and to filter and sort the displayed records.
- To display the dated attendance records for a selected student, with an optional date

filter.

- To save student and attendance records in the browser's Local Storage so they remain available when the application is reopened in the same browser.

## 3. Architecture

The application uses a client-side architecture and runs in a web browser without requiring a separate server or database. HTML defines the dashboard, student form, attendance controls, student directory, and attendance history. CSS controls the layout, colours, cards, tables, status indicators, and responsive behaviour on smaller screens. JavaScript validates input, manages student and attendance records, calculates statistics, supports searching and filtering, and updates the displayed information. Local Storage saves and retrieves the records in the current browser.

When the page opens, JavaScript loads the saved data from Local Storage and displays the dashboard. The user can add or update student details, select an attendance date, choose attendance statuses, and save the entries. The application then recalculates the summary values and refreshes the student directory and attendance history. Search, filter, sort, and low-attendance settings help the user review the records.

**User navigation and application flow**

| |
|---|
| START / OPEN PAGE |
| ↓ |
| Load saved data from Local Storage |
| ↓ |
| Add or edit student details |
| ↓ |
| Choose date and mark attendance |
| ↓ |
| Save attendance and update statistics |
| ↓ |
| Search / filter / sort / view history |
| ↓ |
| END: Attendance dashboard and records |

**Technologies used**

| Technology | |
|---|---|
| HTML5 | Structures forms, cards, tables, buttons, and page sections. |
| CSS3 | Styles the dashboard, layout, responsive behaviour, colours, and status badges. |
| JavaScript | Implements CRUD operations, attendance calculations, filtering, sorting, and history. |
| Local Storage | Stores the project data in the current browser. |

## 4. Practical Implementation

The project is implemented in a single HTML file that contains the HTML structure, CSS styles, and JavaScript logic. The user first enters the student's name, unique student ID, and department in the Add Student form. After a student is added, the attendance section can be used to select a date and mark the student as Present, Absent, Late, or Leave. The Student Directory displays registered students and their attendance information. The Attendance History

section shows the selected student's saved records. JavaScript updates the dashboard after changes and stores the data in Local Storage.

### 4.1 Features Implemented

**Student management:** The user can add a student and edit or delete an existing record. The application checks that the student ID is unique, helping prevent two records from being created with the same ID.

**Daily attendance:** The user selects a date and assigns each student one of four statuses: Present, Absent, Late, or Leave. If attendance for the same student and date is saved again, the existing entry is updated rather than creating a duplicate.

**Bulk marking:** The user can select one attendance status and apply it to all registered students for the selected date. Individual statuses can then be adjusted before saving.

**Statistics dashboard:** The dashboard displays the number of registered students, the number marked Present for the selected day, the number marked Absent, and the overall attendance percentage based on the saved records.

**Attendance percentage:** Present and Late are counted as attended days. Absent is counted as a marked day but not as an attended day. Leave is excluded from the total counted days. The percentage is calculated by dividing the number of attended days by the number of counted days and multiplying by 100.

**Search, filter, and sort:** The user can search by student name, student ID, or department, filter the list by available attendance/status options, and sort the records by name, ID, or attendance percentage.

**Low-attendance warning:** The application compares each student's attendance percentage with the selected warning threshold. Students below the threshold are highlighted. The default threshold is 75%, and it can be changed in the settings.

**Attendance history:** The user selects a student to view that student's attendance records by date. An optional date filter can be used to narrow the displayed history.

**Browser storage:** Student details, attendance records, and the warning threshold are saved in Local Storage. The saved data remains in the same browser profile, but it is not automatically shared with other devices or users.

### 4.2. Source Code

The complete application source code is provided in the GitHub repository linked at the beginning of this report. The program is organised into HTML,

CSS, and JavaScript sections. The HTML creates the page structure and input controls, CSS defines the visual layout, and JavaScript handles validation, attendance operations, calculations, and browser storage. The following examples show important parts of the implementation.

**HTML**

HTML defines the visible structure of the application. The header displays the project name and purpose. The statistics section contains the summary cards. The form collects the student's name, ID, and department. The attendance controls and table sections allow the user to mark attendance and review student records.

These elements are arranged inside the main page so the user can add students, mark attendance, and view the updated information from the dashboard.

**Insert relevant HTML code here.**

```
<header>
        <div class="eyebrow">Student tools · Attendance management</div>
        <h1>Student Attendance Tracker</h1>
 <p>Manage student records, mark daily attendance, and track attendance
percentages.</p>
</header>
<main class="wrap">
        <section class="stats">
 <div class="stat"><span>Total students</span><strong
id="totalStudents">0</strong></div>
 <div class="stat"><span>Present today</span><strong
id="presentToday">0</strong></div>
 <div class="stat"><span>Absent today</span><strong
id="absentToday">0</strong></div>
 <div class="stat"><span>Overall attendance</span><strong
id="overallPercent">0%</strong></div>
        </section>
        <form id="studentForm">
        <label for="studentName">Full name *</label>
        <input id="studentName" required>
        <label for="studentId">Student ID *</label>
        <input id="studentId" required>
        <label for="department">Department *</label>
        <input id="department" required>
```

```
        <button type="submit" id="saveStudent">Add student</button>
        </form>
</main>
```

**CSS:**

CSS defines the appearance and layout of the application. The grid layout arranges the summary cards and dashboard sections. Colours, borders, spacing, and rounded panels make the sections easier to distinguish. Media queries adjust the layout for smaller screens, while attendance status classes use different visual styles to make Present, Absent, Late, and Leave easy to recognise.

The responsive rules change the dashboard columns when the screen becomes narrower, helping the page remain usable on smaller displays.

The status classes provide a consistent visual indicator for each attendance status.

**Insert relevant CSS code here.**

```
* { box-sizing: border-box; }
body {
 margin: 0;
 font-family: "Segoe UI", Arial, sans-serif;
 background: #f4f7fb;
 color: #172033;
}
.stats {
 display: grid;
 grid-template-columns: repeat(4, minmax(0, 1fr));
 gap: 14px;
}
.panel, .stat {
 background: white;
 border: 1px solid #e2e8f0;
 border-radius: 16px;
 box-shadow: 0 5px 18px #0f172a0a;
}
```

```
@media (max-width: 850px) {
 .grid { grid-template-columns: 1fr; }
 .stats { grid-template-columns: repeat(2, 1fr); }
}
```

**JavaScript:**

JavaScript controls the application's behaviour. It loads saved records from Local Storage, validates student details, updates student records, records attendance, calculates percentages, and refreshes the displayed dashboard.

Before saving attendance, the program checks whether an entry already exists for the same student and date.

If an entry exists, its status is updated; otherwise, a new attendance entry is added. This prevents duplicate entries for the same student on the same date.

**Insert relevant JavaScript code here.**

```
const STORAGE_KEY = "studentAttendanceTracker_v1";
let data = loadData();
function loadData() {
 try {
 return JSON.parse(localStorage.getItem(STORAGE_KEY)) ||
 { students: [], attendance: [], threshold: 75 };
 } catch (e) {
 return { students: [], attendance: [], threshold: 75 };
 }
}
function upsertAttendance(studentId, date, status) {
 const existing = attendanceFor(studentId, date);
 if (existing) existing.status = status;
 else data.attendance.push({ studentId, date, status });
}
function statsFor(studentId) {
 const rows = data.attendance.filter(a => a.studentId === studentId);
 const marked = rows.filter(a => a.status !== "Leave");
 const present = marked.filter(
 a => a.status === "Present" || a.status === "Late"
 ).length;
 return {
```

```
 total: marked.length,
 present,
 percent: marked.length
 ? Math.round(present / marked.length * 100) : 0
 };
}
function persist() {
 data.threshold = Number(document.getElementById("threshold").value) ||
75;
 localStorage.setItem(STORAGE_KEY, JSON.stringify(data));
 render();
}
```

### 4.3 Project Execution

• Save the complete source code as student-attendance-tracker.html.

• Open the saved HTML file in a modern web browser, such as Google Chrome or Microsoft Edge.

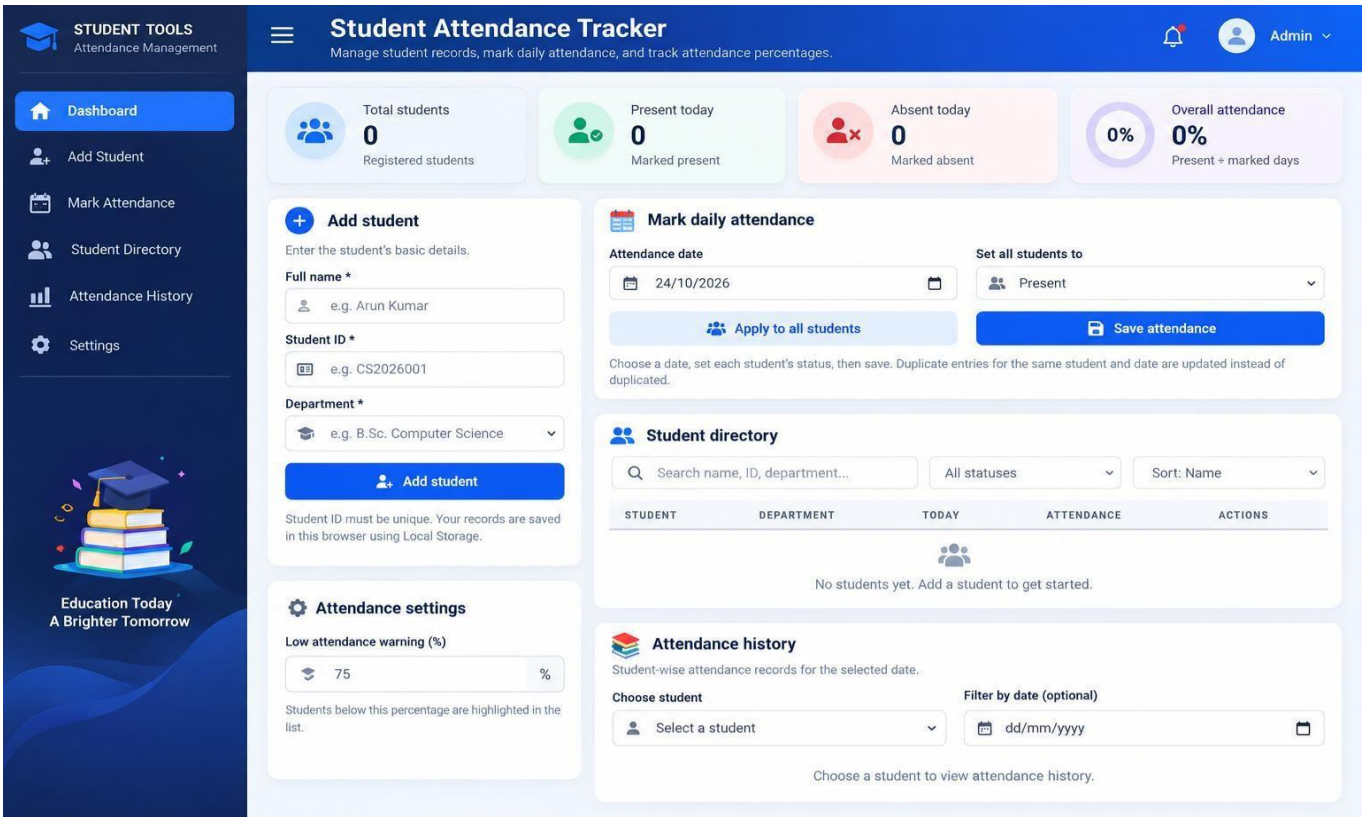
• In the Add Student form, enter the student's full name, a unique student ID, and department, then select Add student.

• Select the attendance date and set each student's status. Use Apply to all students if the same status should be assigned to everyone.

• Select Save attendance. The application saves the entries and refreshes the dashboard statistics and student directory.

• Use the search box, filters, sorting options, and Attendance History section to find and review saved records.

## 5.Output Screenshots:




## 6.Technical Explanation

The Student Attendance Tracker runs in the browser and does not require a server or database installation. JavaScript keeps student details and attendance entries in data structures while the application is open, then saves them as JSON in Local Storage. Each student record contains a unique student ID, name, and department. Each attendance entry connects a student ID with a date and a status. When the user saves or changes records, the application updates the stored data and refreshes the dashboard. Search and filter controls determine which students are shown, while sorting changes their display order. Attendance percentage is calculated by counting Present and Late as attended days, counting Absent as a marked day, and excluding Leave from the total counted days. The low-attendance warning compares the calculated percentage with the threshold selected in Settings. The application also uses form validation, event listeners, and safe handling of user-entered text when displaying records. Local Storage is suitable for this mini project, but data is limited to the current browser profile and is not a central database shared

between users or devices.

## 7.Conclusion

The Student Attendance Tracker provides a convenient way to manage student information and daily attendance in one browser-based application. It supports adding, editing, and deleting student records, marking attendance by date, calculating attendance percentages, searching and sorting records, displaying attendance history, and warning about low attendance. The project demonstrates the practical use of HTML for page structure, CSS for responsive design, JavaScript for application logic, and Local Storage for saving data in the browser.

## 8.Future Enhancements

- Add login and role-based access for staff and administrators.
- Use a database and server so records can be shared across devices.
- Export attendance reports to PDF or CSV.
- Add monthly summaries and date-range reports.
- Add a student photo, class/year filters, and department-level summaries.
- Provide data backup and restore options.
- Improve accessibility and add confirmation messages for important actions.